

Training Laya to Play Better Chess

A step-by-step guide: build a Stockfish-labelled dataset, fine-tune Laya on Kaggle's free GPUs, calibrate it, measure it, and plug it back into your Laya Chess app.

Read this first

Right now the model is weak because Laya's shipped checkpoints are general-purpose and the README itself reports them scoring near chance zero-shot on its own decision benchmark. Fine-tuning on domain data is the intended fix. This guide gets you there, but be realistic: Laya reads the board as text and answers in one forward pass with no search. Expect a decent 'intuition' player that stops hanging pieces and follows opening principles - not an engine. Success here means beating the plain material-greedy baseline, not beating Stockfish.

Rough plan and time

Phase	What you do	Where	Rough time
1-3	Environment, get games, install Stockfish	Your laptop	30-60 min
4-5	Design candidate filter, label positions with Stockfish	Your laptop (CPU)	1-3 h (mostly waiting)
6	Convert to Laya's training format	Your laptop	1 h
7-9	Kaggle setup, smoke test, full fine-tune	Kaggle free 2xT4	README: about 4-5 h for 4 epochs on ~30k questions
10-11	Calibrate, evaluate against baselines	Kaggle + laptop	1 h
12-13	Serve locally, port the filter to the app	Your laptop	1-2 h

Time figures are rough estimates, not measurements. Kaggle GPU quotas and session limits change - check them on Kaggle before planning a long run.

Part A - How the whole thing works

Laya is a non-autoregressive decision model. You give it a state (text) and a fixed list of options, and one forward pass returns a probability for each option. That is why an illegal move can never come out of your app: only legal moves are ever offered as options. Fine-tuning teaches it which option is best *for chess positions specifically*.

The pipeline

#	Stage	Output
1	Lichess games (PGN, CC0 licence)	Real positions from strong players
2	Sample positions, then build the candidate list (top-K legal moves by a cheap heuristic)	State text + up to K options
3	Stockfish scores each candidate	Best move + centipawn score per option
4	Split by GAME into train / calib / test	3 JSONL files
5	Convert to Laya's training format	Kaggle dataset
6	RLCD fine-tune, fit calibration temperatures	Fine-tuned checkpoint
7	Evaluate vs baselines on the untouched test set	Numbers you can trust
8	Serve locally, port the exact same filter + state format to the app	Smarter Laya Chess

The single most common way this goes wrong

Train/inference mismatch. If the app builds the state text, the option labels or the candidate filter even slightly differently from the training data, the model sees inputs it was never trained on and performs like it was never fine-tuned. Part D is a checklist for this. The shared logic lives in one file (candidates.py) so you have a single source of truth to port.

Part B - What you need

- **Python 3.10+** on your laptop, plus `python-chess` and `zstandard` (for .zst PGN files).
- **Stockfish** binary (free, from stockfishchess.org, or your package manager). Any recent version works.
- **A Kaggle account** with phone verification (needed to enable GPUs and internet in notebooks).
- **A Hugging Face account** with a write token (only needed if you want to push the model to the Hub; you can also just download the output from Kaggle).
- **Lichess game database** from database.lichess.org (monthly .pgn.zst files, CC0). You need only a tiny slice - the first 20-50k games is plenty to start.
- **The files shipped with this guide:** `candidates.py`, `gen_dataset.py`, `evaluate.py` (tested against a real Stockfish 16 and a mock Laya server; not yet run against real Laya training).

Things I could not verify - check these yourself

(1) The exact dataset schema the Kaggle notebook expects - I could not open the notebook source. Step 6 tells you how to read it from the notebook itself. (2) Which environment variable or flag points `laya-serve` at your own checkpoint - check `laya/serve.py` in the repo. (3) Whether your installed Laya version includes the newer `python -m laya.finetune` entry point (it appears in a pull request against the repo) or only the notebook script. (4) The maximum state length the model accepts - see Step 4.

Step 1 - Set up your laptop

```
cd laya-chess
mkdir chess_training && cd chess_training          # copy candidates.py, gen_dataset.py, evaluate.py here
python -m venv .venv-train
.venv-train\Scripts\activate          # Windows   (macOS/Linux: source .venv-train/bin/activate)
pip install python-chess zstandard
```

Use a separate virtual environment from the one running laya-serve, so training tools never disturb your working Laya install.

Step 2 - Get games

1. Go to database.lichess.org and download one standard-rated monthly file (.pgn.zst). These files are huge; do not decompress the whole thing.
2. `gen_dataset.py` reads .pgn.zst directly as a stream and stops after `--max-games`, so a partial download is fine if you cancel early and the file is still readable - or simply use a smaller file.
3. Keep only strong games: the script filters with `--min-elo` (default 1800). Higher-rated games give cleaner, more purposeful positions.

Optional but valuable: tactics

The Lichess puzzle database (also CC0) gives positions where the best move is unambiguous. Mixing in around 10-20 percent puzzle positions can sharpen tactical vision. It is a second data source with its own format, so add it after your first end-to-end run works.

Step 3 - Install Stockfish

Download from stockfishchess.org and unzip, or install via your package manager (`apt install stockfish`, `brew install stockfish`). Note the full path to the executable - you pass it to the script. Test it:

```
"C:\tools\stockfish\stockfish.exe"          # should open a prompt; type uci then quit
```

Step 4 - Design the candidate filter and state format

Chess positions average around 30 legal moves, but the Laya README advises keeping choice questions under about 20 options because every option is squeezed into a fixed token budget. So each question offers the top K legal moves (default K = 16) ranked by a cheap heuristic: promotions, then captures (most valuable victim by least valuable attacker), then checks, castling, and centralising moves, with ties broken by move text so it is fully deterministic. If a position has K or fewer legal moves, nothing is dropped.

- **Illegal moves stay impossible.** The filter only ever chooses among legal moves.
- **The filter can drop the truly best move.** `gen_dataset.py` measures this: it prints 'candidate-filter recall' at the end (how often Stockfish's true best move over all legal moves survived the filter). Aim for 90 percent or more. If lower, raise K a little or improve `move_priority()`.
- **State text.** `candidates.py` writes explicit piece lists ('White: Ke1 Qd1 ...'), castling, en passant, and the raw FEN. Explicit piece lists are usually easier for a text model than FEN's run-length digits, but that is a hypothesis, not a fact - test 'FEN only' versus 'piece lists + FEN' on your held-out data.
- **Length check.** The combined state text is a few hundred tokens. Find the state max length used by the notebook (look for `max_len`). If states are truncated, drop the FEN from `state_text()` so the piece lists survive.

Step 5 - Label positions with Stockfish

```
python gen_dataset.py --pgn lichess_2024-xx.pgn.zst ^
--stockfish "C:\tools\stockfish\stockfish.exe" ^
--out data --max-games 6000 --positions-per-game 6 ^
--depth 10 --k 16 --workers 4
```

(Use \ instead of ^ for line continuation on macOS/Linux.) What it does for each sampled position:

1. Builds the top-K candidate list and asks Stockfish to score *each candidate* (multipv restricted to those root moves) at the chosen depth.
2. Also searches all legal moves once to find Stockfish's true best move, only to report filter recall.
3. Skips already-decided positions (best move scored beyond +/-800 centipawns) - labels there are mostly noise. Change with --max-abs-cp.
4. Writes a hard label ('best') and a soft distribution ('soft': each option's weight = $\exp((cp - best_cp) / 100)$, normalised). Soft targets capture that several moves can be nearly equally good.
5. Splits by **game** (hash of the game id) into train 80 / calib 10 / test 10 percent, so positions from one game never appear in two splits.

How much data, and how long

Start with about 5k-10k labelled positions to prove the pipeline, then scale toward 30k or more (the README's own timing example is about 30k questions). Stockfish cost scales with depth and number of candidates, and is CPU-bound and parallel - more --workers means faster, up to your core count. Do a 200-position dry run first and time it before launching the big job.

Sanity check the output: open data/train.jsonl and read a few rows. Does the 'best' move look sensible for the position? Is the option list the size you expect? Are cp values plausible?

Step 6 - Convert to the notebook's training format

This is the one step I can't hand you finished, because I couldn't open the notebook's source. The notebook builds training items with a per-option float 'target' vector, and it trains on a public typed-decisions dataset. Read it yourself:

1. On GitHub, open notebooks/laya_finetune_typed_decisions_2xT4_kaggle.ipynb in the NandhaKishorM/laya repo.
2. Find the cell that builds the training items (search for 'target'). Print one built item.
3. Write a small converter that reads your data/train.jsonl and produces items with the same fields. Use this mapping as your starting point:

Your field	Meaning	Likely use in Laya's item
state	Position text	The text/state the model reads
options[].label	Move text, e.g. 'Nxc4 (capture)'	The option (criteria) text
soft (in option order)	Stockfish-derived probability per option	The float target vector (or use 1.0 for 'best', 0.0 otherwise if the notebook expects hard labels)
best	Stockfish's top pick	Hard label / sanity check

- **Shuffle option order** (fixed seed) when converting, so the model cannot learn position habits. The generator sorts options by move text for reproducibility, which is not something you want the model to memorise.
- The notebook also holds out its own calibration items internally; keep your test.jsonl completely untouched for the final evaluation.
- If you get stuck, paste the notebook's dataset-building cell together with one row of data/train.jsonl into Claude or opencode and ask for the converter.

Step 7 - Set up Kaggle

1. Create/verify your Kaggle account (phone verification).
2. Datasets -> New Dataset: upload your converted train and calib files. Keep it private.
3. Get the notebook: download the .ipynb from the Laya repo, then in Kaggle use File -> Import Notebook (upload).
4. Notebook options: Accelerator = GPU T4 x2 (both GPUs), Internet = On (needed to install packages and download the base checkpoint).
5. Add your dataset to the notebook (Add Input). If you will push to the Hub, add your Hugging Face token under Add-ons -> Secrets - never paste it into a cell.
6. Edit the notebook's dataset cell to load your files instead of the typed-decisions benchmark.

Step 8 - Smoke test before the real run

Run with a tiny slice (for example 500 questions, 1 epoch). You are checking mechanics, not quality:

- It completes without crashing on both GPUs.
- Training reward/loss values change over steps rather than staying constant.
- An output folder is written (the notebook saves to /kaggle/working/...) and the model reloads.
- Sessions can time out. Prefer a notebook version that saves a checkpoint each epoch; if yours doesn't, save one yourself at epoch boundaries.

Step 9 - Full fine-tune

Point the notebook at your full train set and run. The README describes the notebook's method as RLCD - policy-gradient training with proper-scoring-rule rewards - which is designed to reward well-calibrated probabilities, not just correct answers. Watch for the reward improving on the calibration split; if it is flat, revisit your labels and state format before spending more GPU hours.

Symptom	Likely cause	What to try
No improvement at all	Converter bug (targets misaligned with option order)	Verify target vector order matches shuffled option order
Great train reward, poor test	Overfitting or leakage	More positions, fewer epochs; confirm split is by game
Out of memory	State text too long	Drop FEN from state_text(); reduce batch size
Session killed	Kaggle time limit	Resume from last epoch checkpoint; shorten epochs

Step 10 - Calibration

The README notes that the shipped checkpoints are over-confident and that refitting one temperature per question type and option count on held-out data moved mean ECE from 0.466 to 0.081. The notebook fits these temperatures for you after training. Your app's calibration panel exists to show exactly this, so record before-and-after numbers. Fit only on the calib split, never on test.

Step 11 - Evaluate against baselines

Run the same test.jsonl through every predictor. Metrics: top-1 and top-3 agreement with Stockfish, average centipawn loss (ACPL; often more meaningful than top-1 because several moves can be near-equal), blunder rate (moves losing over 200 centipawns), and ECE for probabilistic predictors.

```
python evaluate.py --test data/test.jsonl --predictor random
python evaluate.py --test data/test.jsonl --predictor material
python evaluate.py --test data/test.jsonl --predictor laya --base-url http://localhost:8000 --limit 1000
```

Run the Laya line twice: first with the **base** checkpoint (your 'before'), then with the **fine-tuned** one (your 'after'). Add `--print-raw` once to see the raw server response and fix the parser in `evaluate.py` if your Laya version returns a different shape (the parser is defensive but was only tested against a mock server).

Predictor	What it tells you
random	Floor: about 1/K top-1 (6 percent at K=16)
material (1-ply)	The cheap baseline your calibration panel already uses. Your fine-tuned model must beat this to be worth shipping
Laya base	Your 'before' number
Laya fine-tuned	Your 'after'. Success = lower ACPL and blunder rate than material, and lower ECE than base

Ignore ECE for the material baseline: it outputs 100 percent confidence by construction, so its ECE only reflects how often it is right.

Step 12 - Serve your fine-tuned checkpoint locally

1. Download the output folder from Kaggle (or pull it from the Hub if you pushed it).
2. Open `laya/serve.py` in the repo and find how the server chooses its checkpoint (environment variable or argument). Point it at your folder.
3. Laya includes a router that can pick a checkpoint per request - make sure it isn't silently routing your chess requests to a different, unmodified model. Confirm by checking that `evaluate.py`'s numbers change when you swap checkpoints.
4. Restart `laya-serve` and re-run the `/health` check from `GETTING_STARTED.md`.

Step 13 - Plug it back into the app

Now make the app build questions *exactly* as training did. This is Part D's checklist in action:

- Port `candidates.py` to `frontend/src/chess/engine.ts`: `move_priority`, `candidate_moves` (with K), `state_text`, `option_label` - same numbers, same tie-breaks.
- In `frontend/src/api/layaClient.ts`: send state as `{position: }` (or whatever key your converter used), the same instructions sentence, and criteria mapping `id -> label`, only for the filtered candidates.
- The calibration panel and material check should now compare against the filtered candidate set; the trails/ghost pieces can still show all legal moves if you like, but only candidates get probabilities.
- Play 20-30 moves and compare Laya's picks in the panel with your `evaluate.py` numbers. Big gaps mean a parity bug.

Part D - Train / inference parity checklist

Tick every line before trusting any result from the app:

Item	Training (Python)	App (TypeScript)	Match?
Candidate filter + K	candidate_moves(board, 16)	same logic + K in engine.ts	[]
Tie-break rule	uci text ascending	same	[]
State text format	state_text(board)	identical string, byte for byte	[]
Option label format	SAN + (capture/check/promotion)	identical	[]
Instructions sentence	Which move is best for the side to move in this chess position?	identical	[]
Option ids	UCI (e2e4, e7e8q)	identical, promotion suffix included	[]
Option order	shuffled in training	any order is fine (model saw shuffled)	[]
Endpoint / request shape	/v1/systemone choice question	same	[]

A fast way to check the port: dump 50 random positions, produce state text and candidate lists from both Python and TypeScript, and diff them. They must be identical.

Part E - Iterating once it works

- **More and better data:** scale to 100k+ positions; use depth 12 or more for labels; balance opening, middlegame and endgame; add puzzle positions for tactics.
- **Look at failures:** sort test positions by centipawn loss and read the worst 30. Are they all tactics? All endgames? Queen hangs? Target your next data batch at whatever dominates.
- **Test the state format:** FEN only, piece lists only, piece lists + ASCII board. Same data, three runs, compare ACPL.
- **Safety pre-filter (optional):** before Laya sees the options, drop moves that hang a piece for nothing using a cheap 1-ply check. Mirror it in training. This keeps the 'legal moves only' design and removes a whole class of blunders.
- **Recall vs K:** if filter recall is under 90 percent, raise K toward 20 or refine move_priority(); watch that accuracy doesn't drop as option text gets squeezed.
- **Keep a fixed test set forever** so every experiment is comparable, and log each run's numbers in docs/experiments/.

Part F - Command cheat sheet

```
# 1. generate + label data (laptop)
python gen_dataset.py --pgn games.pgn.zst --stockfish <path> --out data --max-games 6000 \
    --positions-per-game 6 --depth 10 --k 16 --workers 4

# 2. baselines
python evaluate.py --test data/test.jsonl --predictor random
python evaluate.py --test data/test.jsonl --predictor material

# 3. BEFORE fine-tuning: base Laya
laya-serve                                # in the laya venv
python evaluate.py --test data/test.jsonl --predictor laya --limit 1000

# 4. fine-tune on Kaggle (GPU T4 x2, Internet on) -> download output

# 5. AFTER: serve fine-tuned checkpoint, rerun step 3, compare
```

Appendix - candidates.py (shared source of truth)

Port this to TypeScript exactly. The other two scripts (gen_dataset.py, evaluate.py) ship as separate files.

```
import chess

PIECE_VALUE = {chess.PAWN: 1, chess.KNIGHT: 3, chess.BISHOP: 3,
               chess.ROOK: 5, chess.QUEEN: 9, chess.KING: 0}
PIECE_LETTER = {chess.PAWN: "", chess.KNIGHT: "N", chess.BISHOP: "B",
               chess.ROOK: "R", chess.QUEEN: "Q", chess.KING: "K"}

def _centrality(sq: int) -> float:
    f, r = chess.square_file(sq), chess.square_rank(sq)
    return 3.5 - max(abs(f - 3.5), abs(r - 3.5))

def move_priority(board: chess.Board, move: chess.Move) -> float:
    """Cheap heuristic: promotions > captures (MVV-LVA) > checks > centralising moves."""
    score = 0.0
    piece = board.piece_at(move.from_square)
    if move.promotion:
        score += 1000 + PIECE_VALUE[move.promotion]
    if board.is_capture(move):
        victim = PIECE_VALUE[chess.PAWN] if board.is_en_passant(move) else \
            PIECE_VALUE[board.piece_at(move.to_square).piece_type]
        score += 100 + 10 * victim - PIECE_VALUE[piece.piece_type]
    if board.gives_check(move):
        score += 50
    if board.is_castling(move):
        score += 20
    score += _centrality(move.to_square) - _centrality(move.from_square)
    return score

def candidate_moves(board: chess.Board, k: int = 16) -> list[chess.Move]:
    legal = list(board.legal_moves)
    if len(legal) <= k:
        return sorted(legal, key=lambda m: m.uci())
    ranked = sorted(legal, key=lambda m: (-move_priority(board, m), m.uci()))
    return sorted(ranked[:k], key=lambda m: m.uci())

def state_text(board: chess.Board) -> str:
    """Compact, explicit state: side to move + piece lists + castling/en-passant.
    (Piece lists are easier for a text encoder than FEN's run-length digits.
    Compare against plain FEN in your experiments.)"""
    def side(color):
        order = [chess.KING, chess.QUEEN, chess.ROOK, chess.BISHOP, chess.KNIGHT, chess.PAWN]
        parts = []
        for pt in order:
            for sq in sorted(board.pieces(pt, color)):
                parts.append(f"{PIECE_LETTER[pt] or 'P'}{chess.square_name(sq)}")
        return " ".join(parts)
    turn = "White" if board.turn == chess.WHITE else "Black"
    ep = chess.square_name(board.ep_square) if board.ep_square is not None else "-"
    return (f"{turn} to move. White: {side(chess.WHITE)}. Black: {side(chess.BLACK)}. "
           f"Castling: {board.castling_xfen()}. En passant: {ep}. FEN: {board.fen()}")

def option_label(board: chess.Board, move: chess.Move) -> str:
    tags = []
    if board.is_capture(move):
        tags.append("capture")
    if board.gives_check(move):
        tags.append("check")
    if move.promotion:
        tags.append("promotion")
    san = board.san(move)
    return f"{san} ({', '.join(tags)})" if tags else san
```